Departamento de
Tecnología Digital

Tecsup TECNOLOGÍA CON SENTIDO



LABORATORIO N° 08

Desarrollo de Aplicaciones Empresariales Avanzadas

DOCENTE:

Arévalo Sermeño, Edwin William

CURSO:

Desarrollo de Aplicaciones Empresariales Avanzadas
6 - C24 - Sección C - D

MVC con Dapper SEMANA08

En este desafío, se te pide que diseñes e implementes un programa con las siguientes partes:

Base de Datos:

1. Reutilizar la base de datos BibliotecaDB de la semana anterior, con sus tablas y datos de prueba. No se crea una base de datos nueva.
2. Crear los procedimientos almacenados para libros: listado de libros activos con el nombre del autor (INNER JOIN con Autores), búsqueda por título, obtención por Id e inserción, actualización y eliminación. **La eliminación es lógica (Activo = 0), nunca un DELETE físico.**
3. Crear los procedimientos almacenados para socios (listado de activos e inserción), el listado de autores activos (para la lista desplegable) y el reporte de préstamos por intervalo de fechas, con INNER JOIN entre Prestamos, DetallePrestamo, Libros y Socios.

ASP.NET Core MVC:

4. Crear un proyecto **ASP.NET Core Web App (Model-View-Controller)** de nombre Biblioteca.Web y agregar los paquetes NuGet Dapper y Microsoft.Data.SqlClient. Todo el código vive en este único proyecto, sin referenciar proyectos de semanas anteriores.
5. Guardar la cadena de conexión en el archivo appsettings.json y leerla mediante IConfiguration; no debe estar escrita en el código.
6. Models: crear las clases Libro, Socio y PrestamoReporte, y validar los campos del libro con DataAnnotations ([Required], [StringLength], [Range] para los ejemplares y [DataType] donde corresponda).
7. Acceso a datos con Dapper: crear en una carpeta Repositorios las clases LibroRepositorio y SocioRepositorio, que ejecuten los procedimientos con QueryAsync y ExecuteAsync usando commandType: CommandType.StoredProcedure y parámetros. Registrarlas en Program.cs e inyectarlas por constructor. **Los controladores no contienen SQL ni abren conexiones.**
8. LibrosController: acciones Index (listado con búsqueda por título), Details, Create, Edit y Delete, con sus vistas Razor en Views/Libros. Usar ViewData["Title"] y vistas fuertemente tipadas con @model. Los formularios de Create y Edit incluyen una lista desplegable de autores.
9. Create, Edit y Delete: la acción GET muestra la vista y la acción POST la procesa con ModelState.IsValid **Después de guardar se redirige (patrón Post/Redirect/Get)** y el resultado se muestra con TempData.
10. SociosController: acciones Index y Create con sus vistas. Si el DNI ya existe, mostrar un mensaje de validación en el formulario (ModelState.AddModelError) en lugar de dejar que la página falle.
11. PrestamosController: acción Reporte con filtro de fechas enviado por GET (desde y hasta), que muestre la tabla de préstamos (socio, libros, fecha límite y estado) y conserve el rango elegido en ViewData.

Más requerimientos:

12. Todas las acciones que acceden a datos deben ser async Task<IActionResult>, con async/await de punta a punta; no usar .Result ni .Wait().
13. Modificar _Layout.cshtml para incluir un menú con los enlaces a Libros, Socios y Reporte de préstamos.
14. Opcional (punto extra): crear una vista parcial (partial view) para la fila de libro y reutilizarla en el listado y en los resultados de la búsqueda.

Adjuntas scripts con la entrega de este documento.

Repositorio:

Capturas de las vistas:

Explicación: **Debe describir el recorrido completo de una petición (ruta, controlador, repositorio, procedimiento almacenado y vista) para una acción a elección, e indicar si usó ViewData, TempData o el modelo para pasar datos a la vista y por qué.**

Observaciones y Conclusiones: